

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
import pickle

from sklearn.linear_model import Ridge
from sklearn.metrics import mean_squared_error, r2_score
import pickle
import numpy as np

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np
import matplotlib.pyplot as plt
```

```
df = pd.read_csv("/content/drive/MyDrive/forestfires.csv")

print("Dataset Loaded Successfully")
print(df.head())
```

```
Dataset Loaded Successfully
   x  y month  day  ffmc  dmc   dc  isi  temp  rh  wind  rain  area
0  7  5   mar  fri  86.2  26.2  94.3  5.1   8.2  51   6.7   0.0   0.0
1  7  4   oct  tue  90.6  35.4  669.1  6.7  18.0  33   0.9   0.0   0.0
2  7  4   oct  sat  90.6  43.7  686.9  6.7  14.6  33   1.3   0.0   0.0
3  8  6   mar  fri  91.7  33.3  77.5  9.0   8.3  97   4.0   0.2   0.0
4  8  6   mar  sun  89.3  51.3  102.2  9.6  11.4  99   1.8   0.0   0.0
```

```
print("Shape of Dataset:")
print(df.shape)
```

```
Shape of Dataset:
(517, 13)
```

```
print("Dataset Information:")
print(df.info())
```

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 517 entries, 0 to 516
Data columns (total 13 columns):
 #   Column  Non-Null Count  Dtype
---  -
0    x      517 non-null      int64
1    y      517 non-null      int64
2   month  517 non-null      object
3    day   517 non-null      object
4   ffmc   517 non-null      float64
5    dmc   517 non-null      float64
6    dc    517 non-null      float64
7    isi   517 non-null      float64
8   temp   517 non-null      float64
9    rh    517 non-null      int64
10   wind   517 non-null      float64
11   rain   517 non-null      float64
12   area   517 non-null      float64
dtypes: float64(8), int64(3), object(2)
memory usage: 52.6+ KB
None
```

```
print("Columns in Dataset:")
print(df.columns)
```

```
Columns in Dataset:
Index(['x', 'y', 'month', 'day', 'ffmc', 'dmc', 'dc', 'isi', 'temp', 'rh',
       'wind', 'rain', 'area'],
      dtype='object')
```

```
print("Missing Values in Dataset:")
print(df.isnull().sum())
```

```
Missing Values in Dataset:
x      0
y      0
month  0
day    0
ffmc   0
```

```

dmc      0
dc        0
isi       0
temp      0
rh         0
wind      0
rain      0
area      0
dtype: int64

```

```

print("Statistical Summary:")
print(df.describe())

```

Statistical Summary:

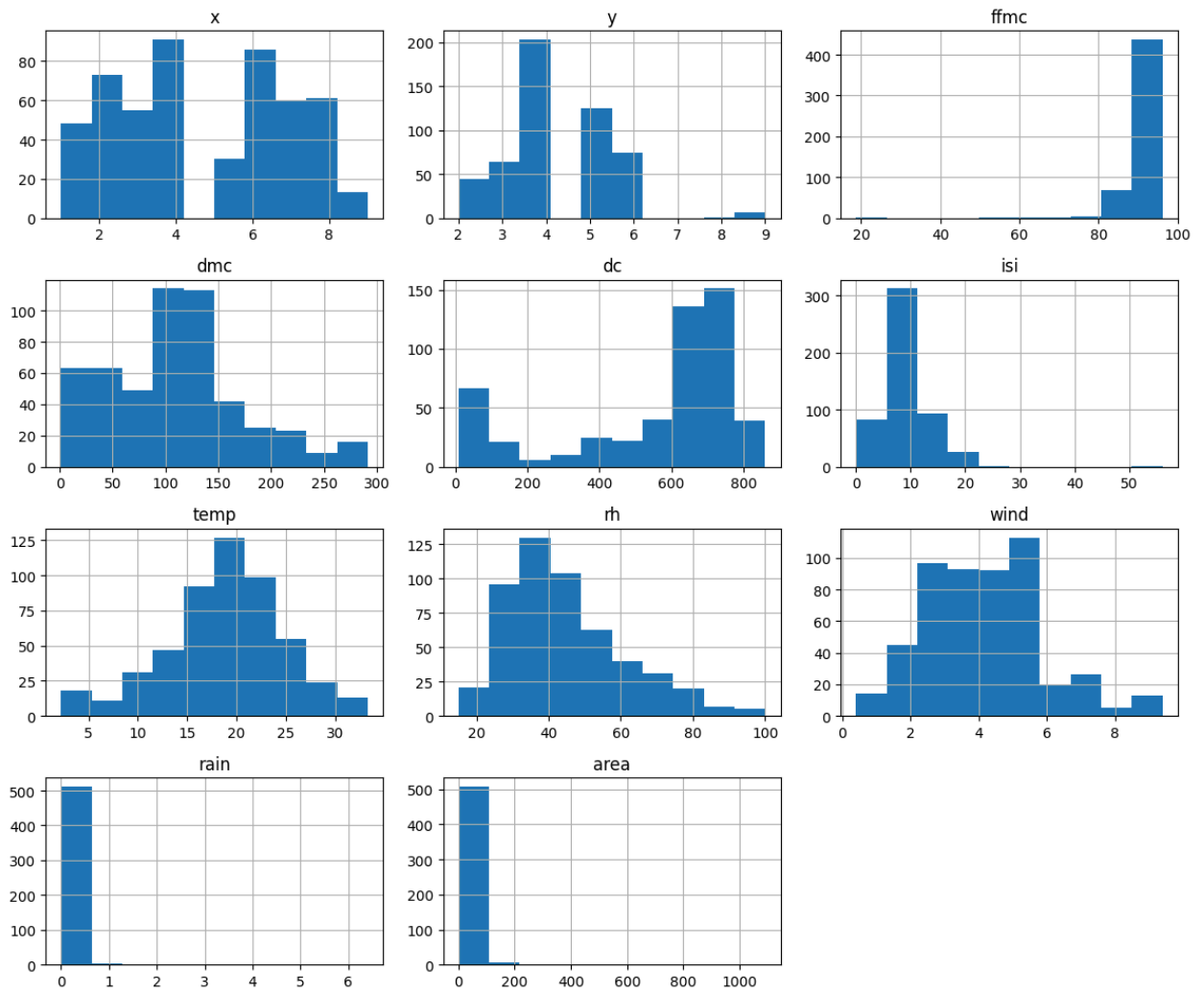
	x	y	ffmc	dmc	dc	isi \
count	517.000000	517.000000	517.000000	517.000000	517.000000	517.000000
mean	4.669246	4.299807	90.644681	110.872340	547.940039	9.021663
std	2.313778	1.229900	5.520111	64.046482	248.066192	4.559477
min	1.000000	2.000000	18.700000	1.100000	7.900000	0.000000
25%	3.000000	4.000000	90.200000	68.600000	437.700000	6.500000
50%	4.000000	4.000000	91.600000	108.300000	664.200000	8.400000
75%	7.000000	5.000000	92.900000	142.400000	713.900000	10.800000
max	9.000000	9.000000	96.200000	291.300000	860.600000	56.100000

	temp	rh	wind	rain	area
count	517.000000	517.000000	517.000000	517.000000	517.000000
mean	18.889168	44.288201	4.017602	0.021663	12.847292
std	5.806625	16.317469	1.791653	0.295959	63.655818
min	2.200000	15.000000	0.400000	0.000000	0.000000
25%	15.500000	33.000000	2.700000	0.000000	0.000000
50%	19.300000	42.000000	4.000000	0.000000	0.520000
75%	22.800000	53.000000	4.900000	0.000000	6.570000
max	33.300000	100.000000	9.400000	6.400000	1090.840000

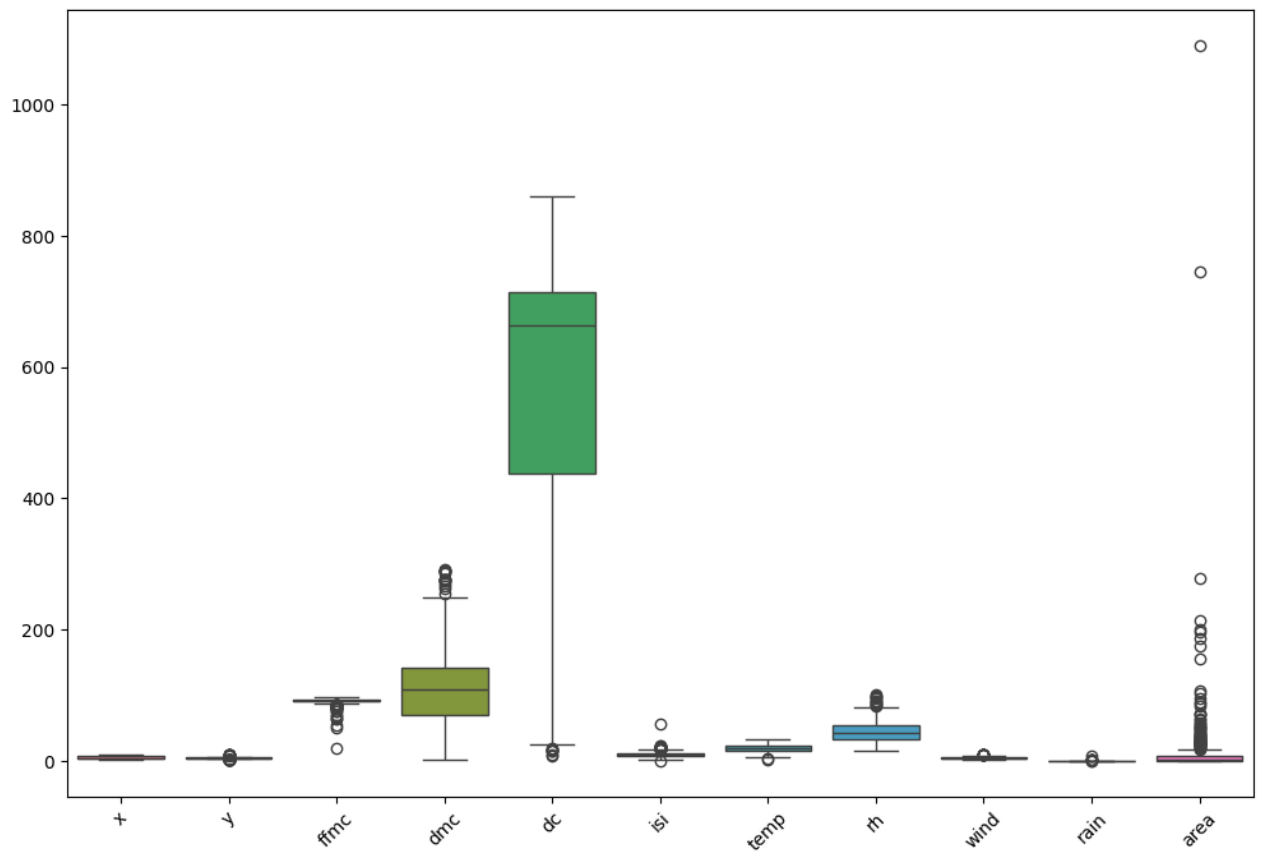
```

df.hist(figsize=(12,10))
plt.tight_layout()
plt.show()

```



```
plt.figure(figsize=(12,8))
sns.boxplot(data=df)
plt.xticks(rotation=45)
plt.show()
```



```
print("Missing Values in Dataset:")
print(df.isnull().sum())
```

Missing Values in Dataset:

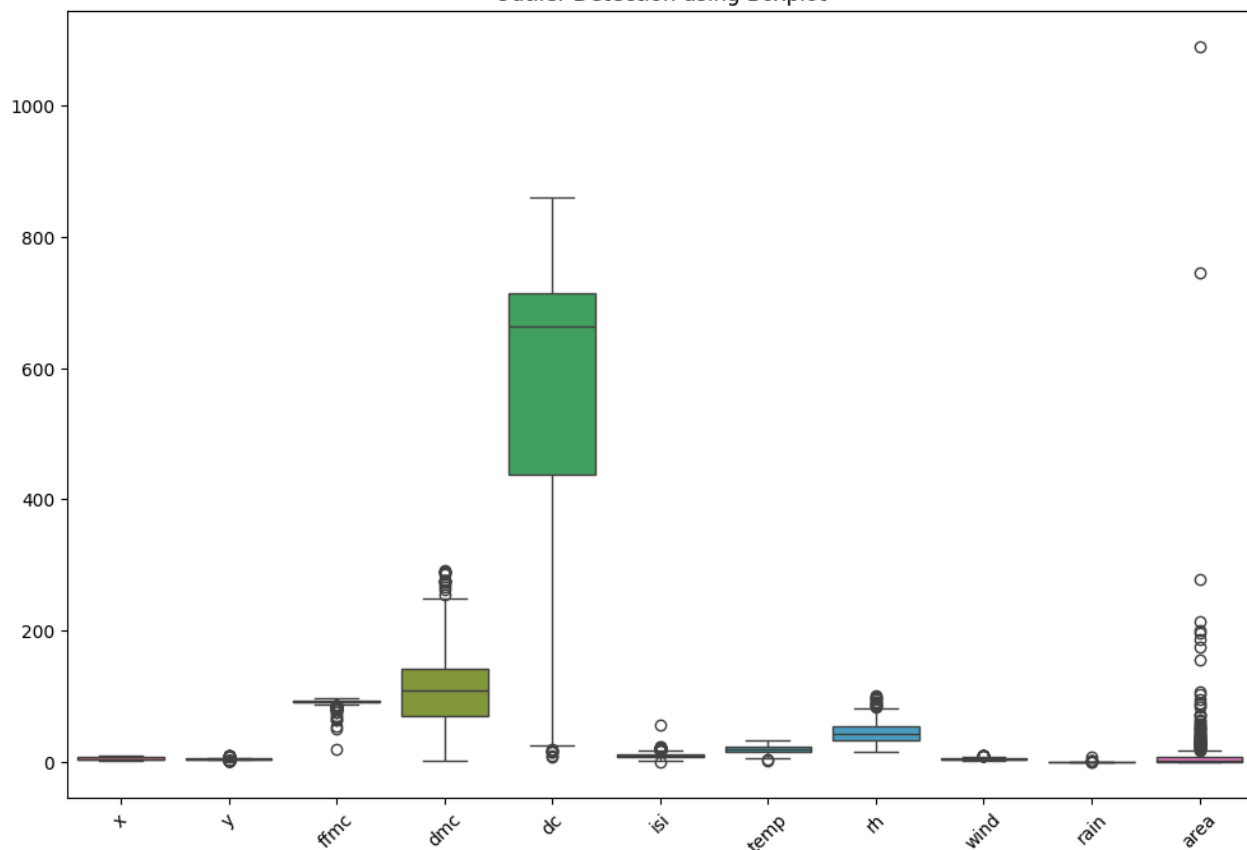
```
x      0
y      0
month  0
day    0
ffmc   0
dmc    0
dc     0
isi    0
temp   0
rh     0
wind   0
rain   0
area   0
dtype: int64
```

```
df = df.dropna()
```

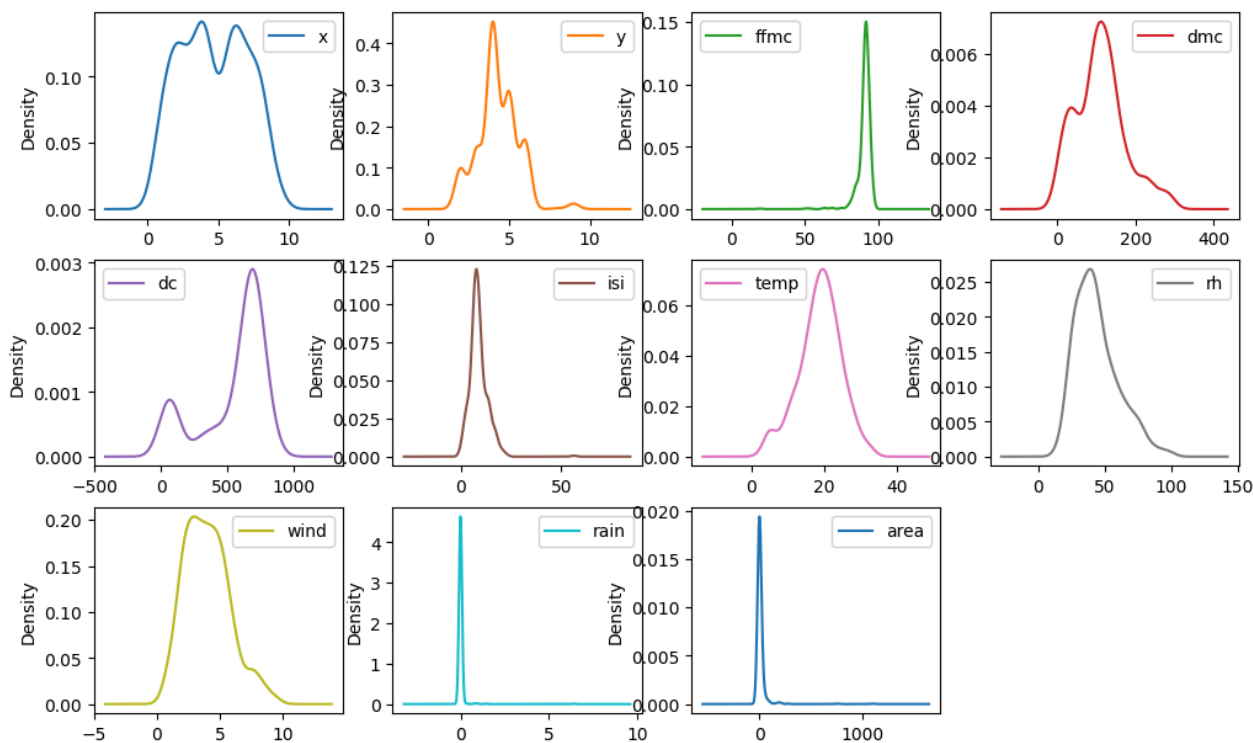
```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(12,8))
sns.boxplot(data=df)
plt.xticks(rotation=45)
plt.title("Outlier Detection using Boxplot")
plt.show()
```

Outlier Detection using Boxplot



```
df.plot(kind='density', subplots=True, layout=(4,4), figsize=(12,10), sharex=False)
plt.show()
```



```
from sklearn.preprocessing import LabelEncoder
import seaborn as sns
import matplotlib.pyplot as plt

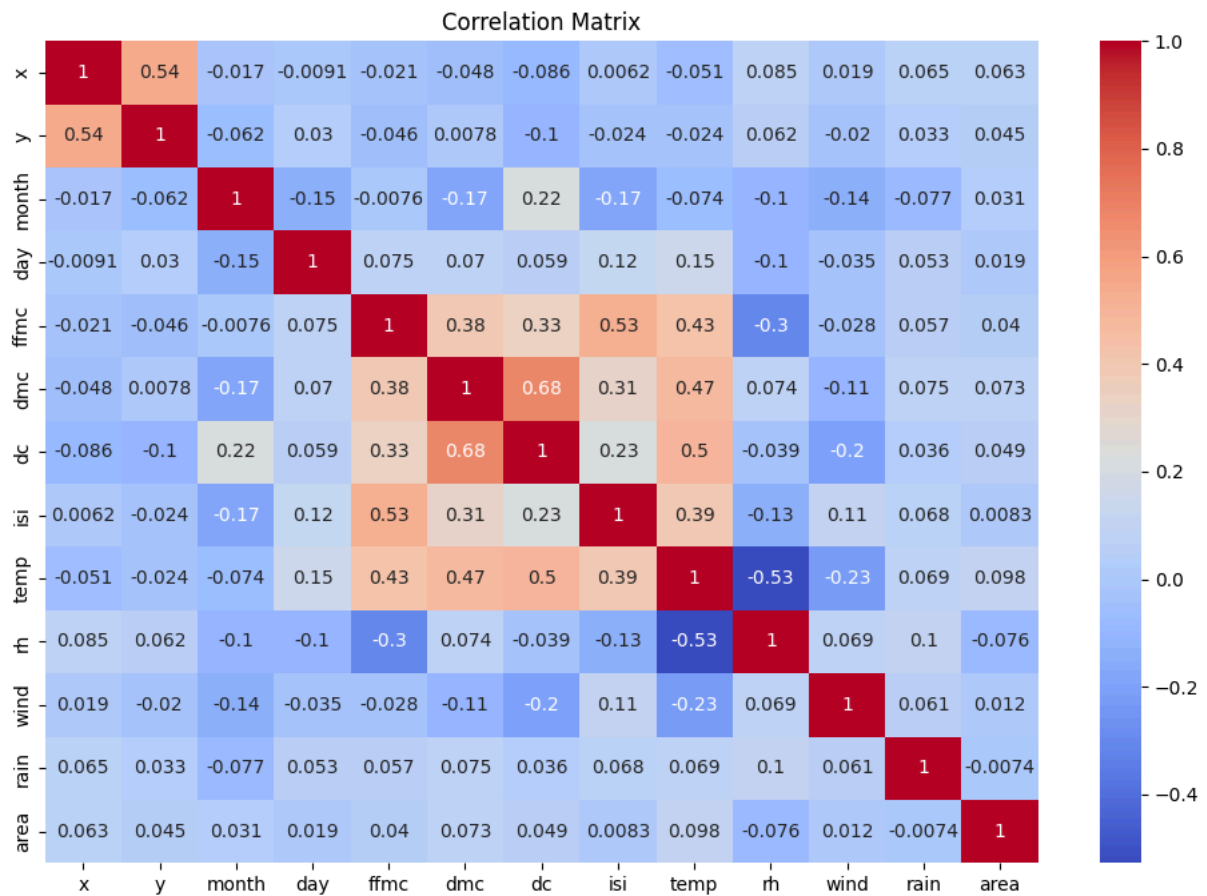
# Encode categorical columns
le = LabelEncoder()
df['month'] = le.fit_transform(df['month'])
```

```
df['day'] = le.fit_transform(df['day'])

# Correlation matrix
plt.figure(figsize=(12,8))
corr = df.corr()

sns.heatmap(corr, annot=True, cmap='coolwarm')

plt.title("Correlation Matrix")
plt.show()
```



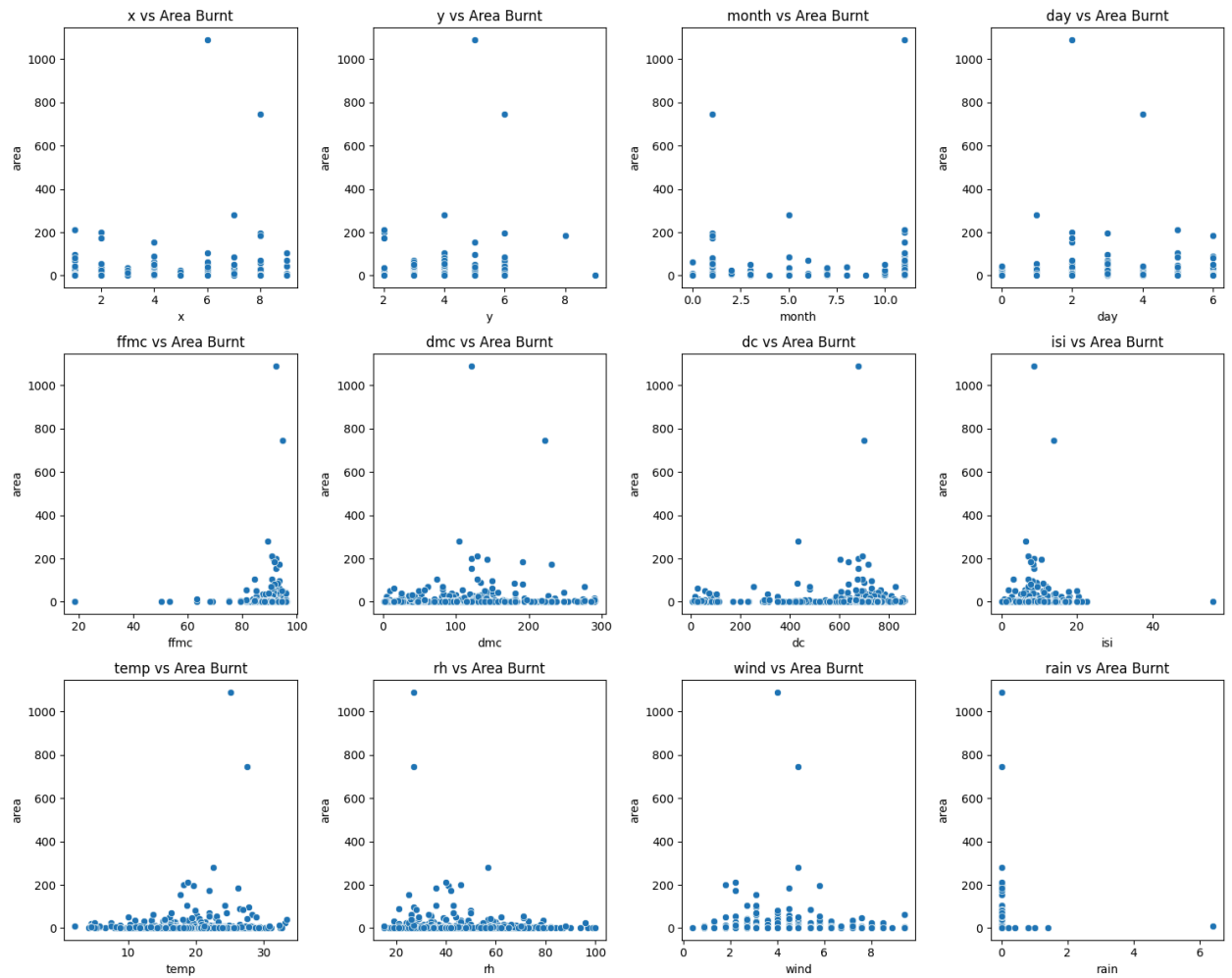
```
import math

cols = [col for col in numeric_cols if col != "area"]

ncols = math.ceil(len(cols) / 3)
fig, axes = plt.subplots(3, ncols, figsize=(15, 12))
axes = axes.flatten()

for i, col in enumerate(cols):
    sns.scatterplot(x=df[col], y=df["area"], ax=axes[i])
    axes[i].set_title(f"{col} vs Area Burnt")

plt.tight_layout()
plt.show()
```



```
from sklearn.preprocessing import LabelEncoder
```

```
le = LabelEncoder()
```

```
df['month'] = le.fit_transform(df['month'])
```

```
df['day'] = le.fit_transform(df['day'])
```

```
df.to_csv("cleaned_forestfires.csv", index=False)
```

```
print("Cleaned dataset saved successfully.")
```

```
Cleaned dataset saved successfully.
```

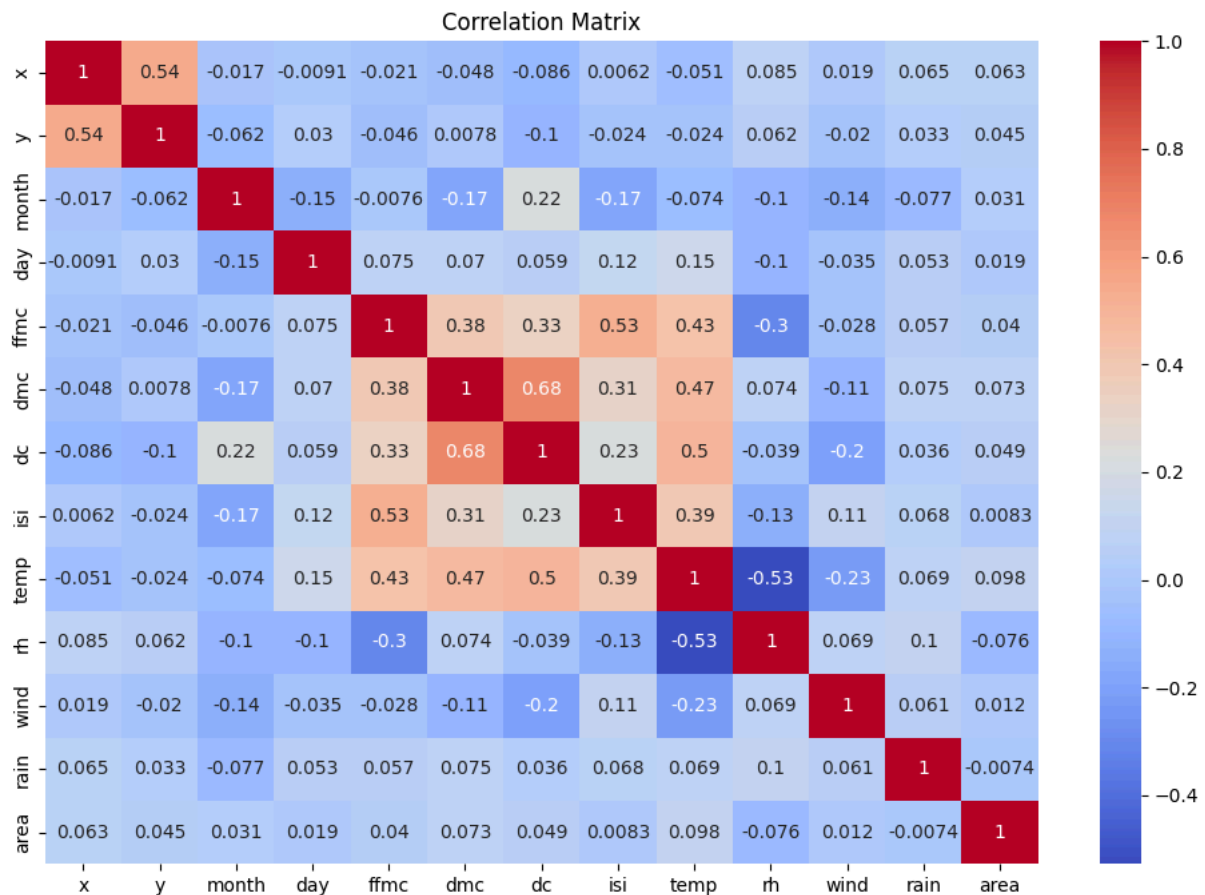
```
le = LabelEncoder()
```

```
df['month'] = le.fit_transform(df['month'])
```

```
df['day'] = le.fit_transform(df['day'])
```

```
corr = df.corr()
```

```
plt.figure(figsize=(12,8))
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Correlation Matrix")
plt.show()
```



```
features = df.drop('area', axis=1).columns
```

```
X = df[features]
y = df['area']
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

```
print("Training Data Shape:", X_train.shape)
print("Testing Data Shape:", X_test.shape)
```

```
Training Data Shape: (413, 12)
Testing Data Shape: (104, 12)
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
with open("scaler.pkl", "wb") as f:
    pickle.dump(scaler, f)
```

```
print("Scaler saved successfully")
```

```
Scaler saved successfully
```

```
print("Sample Scaled Data:")
print(X_train_scaled[:5])
```



```

Sample Scaled Data:
[[-2.88472000e-01 -1.09485913e+00  1.17005658e+00 -3.77415676e-01
   2.80285578e-01 -1.50281988e-01  8.10271937e-01 -9.58383543e-02
   7.97602401e-01 -1.05941934e+00  1.63124094e-03 -6.78170911e-02]
 [-2.88472000e-01 -2.66696454e-01  1.17005658e+00 -9.02239496e-01
   6.16522396e-02  2.17866338e-01  5.49136140e-01 -4.55434308e-01
  -2.05272404e-01 -3.35214892e-01 -1.00892252e+00 -6.78170911e-02]
 [-1.16128472e+00  5.61466218e-01 -1.12340981e+00  1.19705578e+00
   2.63467629e-01  6.14918871e-01  4.35441631e-01  1.41960174e+00
   2.27001219e-01  1.47588074e-01  1.63124094e-03 -6.78170911e-02]
 [-7.24878358e-01 -2.66696454e-01 -1.12340981e+00  1.19705578e+00
   9.36185594e-01  1.04848198e+00  4.87668790e-01  1.41960174e+00
   2.31920555e+00 -1.05941934e+00 -1.00892252e+00 -6.78170911e-02]
 [ 1.47934359e-01 -2.66696454e-01 -6.64716530e-01  1.47408143e-01
  -6.27883674e-01 -1.46922661e+00 -2.01481564e+00 -1.25168249e+00
  -1.12169248e+00  5.09690297e-01 -1.00892252e+00 -6.78170911e-02]]

```

```

ridge = Ridge(alpha=1.0)

ridge.fit(X_train_scaled, y_train)

```

▼ Ridge ⓘ ?

Ridge()

```

alphas = [0.01, 0.1, 1, 10, 100]

for a in alphas:

    model = Ridge(alpha=a)
    model.fit(X_train_scaled, y_train)

    y_pred = model.predict(X_test_scaled)

    r2 = r2_score(y_test, y_pred)

    print("Alpha:", a, "R2 Score:", r2)

```

```

Alpha: 0.01 R2 Score: 0.003292989382857048
Alpha: 0.1 R2 Score: 0.003291469052565832
Alpha: 1 R2 Score: 0.0032762148866374297
Alpha: 10 R2 Score: 0.0031205420029297715
Alpha: 100 R2 Score: 0.001762685443910783

```

```

ridge = Ridge(alpha=1)

ridge.fit(X_train_scaled, y_train)

```

▼ Ridge ⓘ ?

Ridge(alpha=1)

```

y_train_pred = ridge.predict(X_train_scaled)
y_test_pred = ridge.predict(X_test_scaled)

train_r2 = r2_score(y_train, y_train_pred)
test_r2 = r2_score(y_test, y_test_pred)

train_rmse = np.sqrt(mean_squared_error(y_train, y_train_pred))
test_rmse = np.sqrt(mean_squared_error(y_test, y_test_pred))

print("Training R2 Score:", train_r2)
print("Testing R2 Score:", test_r2)

print("Training RMSE:", train_rmse)
print("Testing RMSE:", test_rmse)

```

```

Training R2 Score: 0.03518590516826148
Testing R2 Score: 0.0032762148866374297
Training RMSE: 44.79375786372111
Testing RMSE: 108.39346326679504

```

```

with open("ridge.pkl", "wb") as f:
    pickle.dump(ridge, f)

print("Ridge Regression model saved successfully")

```

Ridge Regression model saved successfully